

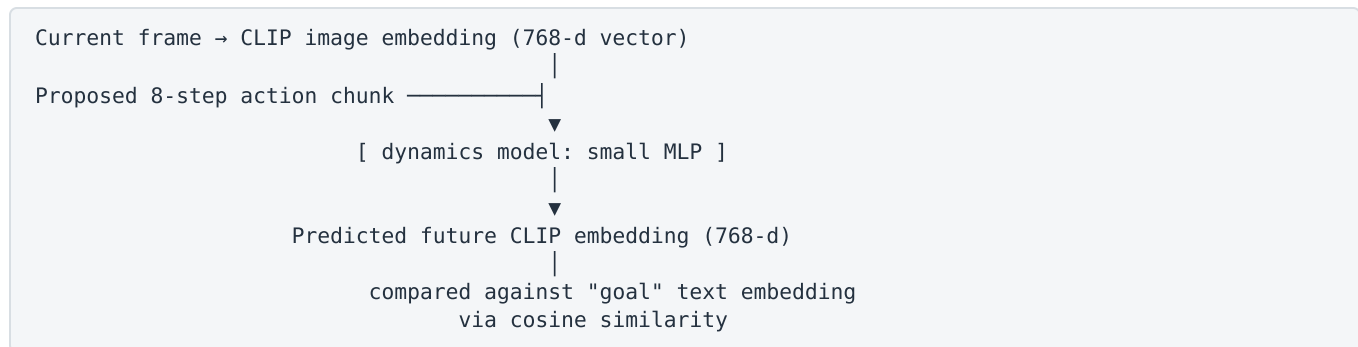
Cross-Modal Latent World Models

Latent dynamics model for block-pushing on the LangTable simulator — core idea, training, and end-to-end fine-tuning

The Core Idea: Predict Meaning, Not Pixels

Rather than predicting future camera frames (expensive, and full of irrelevant visual detail), the model predicts a future **CLIP embedding** — a fixed-length vector that CLIP (a general-purpose vision-language model from OpenAI/LAION) produces to summarize the semantic content of an image or a sentence, such that semantically similar images and sentences land near each other in the same vector space. The dynamics model's job is: given the CLIP embedding of the *current* camera frame, plus a short chunk of proposed push actions, predict the CLIP embedding of the frame *several steps in the future*, after those pushes are executed.

Because goal instructions ("push X close to Y") can also be turned into CLIP text embeddings, the planner never needs a rendered image at all during planning — it can compare the model's predicted future embedding directly against a goal-text embedding using simple vector similarity (cosine similarity), and pick whichever candidate action sequence gets closest.



How the Model Is Trained: Contrastive Learning on Scene Statements

The dynamics model isn't trained to reproduce an exact target vector (a plain regression loss would encourage a "blurry average" prediction and give no signal about which differences in the scene actually matter). Instead it's trained **contrastively**: for a given predicted embedding, the training objective simply asks the model to rank one correct description of the resulting scene above a large pool of incorrect ones.

1. Turning the scene into text

For every timestep of every demonstration, the exact positions of the blocks and the peg are known from the simulator, so a short factual sentence can be generated automatically, e.g.:

"The green star is touching the green cube. The peg is not touching the green star."

This statement is CLIP-text-encoded into the same 768-dimensional space as the image embeddings. That becomes the **positive** (correct) target for whatever frame occurs at that timestep.

2. Manufacturing guaranteed-false negatives

For the same timestep, the training code also generates a large batch of statements about the *same* blocks that are guaranteed to be false — e.g. claiming two blocks are touching when they are known not to be, or that the peg is touching a block it demonstrably isn't. With 8 blocks on the table, there are 56 ordered block pairs and 8 possible "peg touching" clauses, giving several hundred combinatorially-generated false statements per timestep (224 in the original scheme; 448 in the current, wider variant covering both fine-tuned CLIP towers). These are the **hard negatives**.

3. The contrastive (InfoNCE) loss

Given the model's predicted embedding, the training loss (a standard contrastive objective called InfoNCE) treats this as a classification problem: out of one correct statement and hundreds of manufactured false ones, the model must assign the highest similarity score to the correct one. This pushes the predicted embedding toward the true future

scene description and away from every plausible-sounding false alternative, without ever requiring an expensive dense regression target.

Fine-Tuning CLIP End-to-End

The newer line of work unfreezes CLIP's own image encoder *and* text encoder and trains them jointly with the MLP, all under the same contrastive loss described above. The image encoder, the MLP, and the text encoder are updated together by the same gradient signal, rather than treating CLIP as a fixed, generic feature extractor. The idea is that CLIP's off-the-shelf embeddings weren't designed to capture this specific task's fine-grained notion of "touching" — end-to-end fine-tuning lets the whole pipeline specialize its representation of the scene around exactly the distinctions the planner cares about.

This variant is more expensive to train than a frozen-CLIP baseline: because the text encoder is now trainable, its embeddings can no longer be pre-computed and cached — every one of the several hundred candidate statements has to be re-encoded on the fly, every training step, for both towers simultaneously. It has also been extended to train on a combined dataset: the original ~5,000 successful demonstration episodes plus roughly 1,675 additional "suboptimal" episodes (mostly failed attempts), so the model sees a wider range of scene configurations, not just ones that end in success.

At planning time, the trained model's predictions are used by an MPPI-style sampling planner, which scores candidate action sequences by their predicted closeness to the goal and executes the best one.